

Sam

后缀自动机 (SAM)

一些前置约定/定义

记 Σ 为字符集， $|\Sigma|$ 为字符集大小。对于一个字符串 s ，记 $|s|$ 为其长度。

后缀自动机概述

后缀自动机(suffix automaton, SAM) 是一个能解决许多字符串相关问题的有力的数据结构。

举个例子，以下的字符串问题都可以在线性时间内通过 SAM 解决。

- 在另一个字符串中搜索一个字符串的所有出现位置。
- 计算给定的字符串中有多少个不同的子串。

直观上，字符串的 SAM 可以理解为给定字符串的 **所有子串** 的压缩形式。值得注意的事实是，SAM 将所有的这些信息以高度压缩的形式储存。对于一个长度为 n 的字符串，它的空间复杂度仅为 $O(n)$ 。此外，构造 SAM 的时间复杂度仅为 $O(n)$ 。准确地说，一个 SAM 最多有 $2n$ 个节点和 $3n$ 条转移边。

定义

字符串 s 的 SAM 是一个接受 s 的所有后缀的最小 **DFA**（确定性有限自动机或确定性有限状态自动机）。

换句话说：

- SAM 是一张有向无环图。结点被称作 **状态**，边被称作状态间的 **转移**。
- 图存在一个源点，称作 **初始状态**，其它各结点均可从 **初始状态** 出发到达。
- 每个 **转移** 都标有一些字母。从一个结点出发的所有转移均 **不同**。
- 存在一个或多个 **终止状态**。如果从初始状态 **出发**，最终转移到了一个终止状态，则路径上的所有转移连接起来一定是字符串 s 的一个后缀。每个后缀均可用一条从 **初始状态** 到某个终止状态的路径构成。
- 在所有满足上述条件的自动机中，SAM 的结点数是最少的。

子串的性质

SAM 最简单、也最重要的性质是，它包含关于字符串 s 的所有子串的信息。任意从初始状态 **开始** 的路径，如果我们将转移路径上的标号写下来，都会形成 s 的一个 **子串**。反之每个 s 的子串对应从 **初始状态** 开始的某条路径。

为了简化表达，我们称子串 **对应** 一条路径（从 **初始状态** 开始、由一些标号构成这个子串）。反过来，我们说任意一条路径 **对应** 它的标号构成的字符串。

到达某个状态的路径可能不止一条，因此我们说一个状态对应一些字符串的集合，这个集合的每个元素对应这些路径。

构建过程

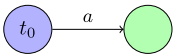
我们将会在这里展示一些简单的字符串的后缀自动机。

我们用蓝色表示初始状态，用绿色表示终止状态。

对于字符串：



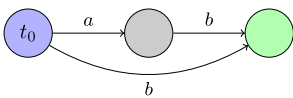
对于字符串：



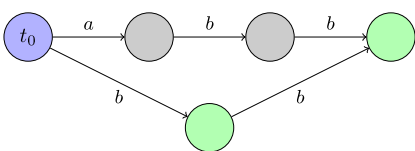
对于字符串：



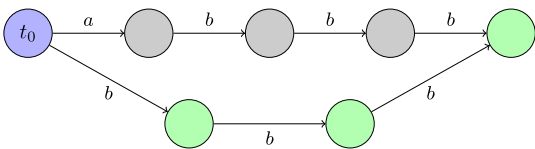
对于字符串：



对于字符串：



对于字符串：



在线性时间内构造

在我们描述线性时间内构造 SAM 的算法之前，我们需要引入几个对理解构造过程非常重要的概念，并对其性质进行简单证明。

结束位置 `endpos`

考虑字符串 的任意非空子串，我们记 为在字符串 中的所有结束位置（假设对字符串中字符的编号从零开始）。例如，对于字符串，我们有。

两个子串 与 的 集合可能相等：。这样所有字符串 的非空子串都可以根据它们的 集合被分为若干 等价类。

显然，SAM 中的每个状态对应一个或多个 相同的子串。换句话说，SAM 中的状态数等于所有子串的等价类的个数，再加上初始状态。SAM 的状态个数等价于 相同的一个或多个子串所组成的集合的个数。

我们稍后将会用这个假设来介绍构造 SAM 的算法。我们将发现，SAM 需要满足的所有性质，除了最小性以外都满足了。由 Nerode 定理我们可以得出最小性（不会在这篇文章中证明）。

由 的值我们可以得到一些重要结论：

引理 1: 字符串 u 的两个非空子串 x 和 y (假设 $x \neq y$) 的 LCP 相同, 当且仅当字符串 u 在 SAM 中的每次出现, 都是以 x 或 y 的形式存在的。

引理显然成立。如果 x 和 y 的 LCP 相同, 则 x 是 y 的一个后缀, 且只以 y 中的一个 x 的后缀的形式出现。且根据定义, 如果 y 是 x 的一个后缀, 且只以后缀的形式在 u 中出现时, 两个子串的 LCP 相同。

引理 2: 考虑两个非空子串 x 和 y (假设 $x \neq y$)。那么要么 x 是 y 的一个后缀, 要么 y 是 x 的一个后缀, 取决于 x 和 y 的 LCP 是否等于 x 或 y 的长度。

证明: 如果集合 S_x 与 S_y 有至少一个公共元素, 那么由于字符串 x 与 y 在相同位置结束, x 是 y 的一个后缀。所以在每次 x 出现的位置, 子串 y 也会出现。所以 y 是 x 的一个后缀。

引理 3: 考虑一个等价类, 将类中的所有子串按长度非递增的顺序排序。每个子串都不会比它前一个子串长, 与此同时每个子串也是它前一个子串的后缀。换句话说, 对于同一等价类的任一子串, 较短者为较长者的后缀, 且该等价类中的子串长度恰好覆盖整个区间 $[1, \text{len}(u)]$ 。

▼ 证明

如果等价类中只包含一个子串, 引理显然成立。现在我们来讨论子串元素个数大于 1 的等价类。

由引理 1, 两个不同的等价类的字符串中, 较短者总是较长者的真后缀。因此, 等价类中没有等长的字符串。

记 u 为等价类中最长的字符串、 v 为等价类中最短的字符串。由引理 1, 字符串 v 是字符串 u 的真后缀。现在考虑长度在区间 $[1, \text{len}(u)]$ 中的任意后缀。容易看出, 这个后缀也在同一等价类中, 因为这个后缀只能在字符串 u 中以 v 的一个后缀的形式存在 (也因为较短的后缀在 u 中只以 v 的后缀的形式存在)。因此, 由引理 1, 这个后缀和字符串 u 的 LCP 相同。

后缀链接 link

考虑 SAM 中某个不是初始状态的状态 p 。我们已经知道, 状态 p 对应于具有相同 LCP 的等价类。我们如果定义 $\text{link}(p)$ 为这些字符串中最长的一个, 则所有其它的字符串都是 p 的后缀。

我们还知道字符串 u 的前几个后缀 (按长度降序考虑) 全部包含于这个等价类, 且所有其它后缀 (至少有一个——空后缀) 在其它的等价类中。我们记 v 为最长的这样的后缀, 然后将 p 的后缀链接连到 v 上。

换句话说, 一个 **后缀链接** 连接到对应于 u 的最长后缀的另一个等价类的状态。

以下我们假设初始状态对应于它自己这个等价类 (只包含一个空字符串)。为了方便, 我们规定 $\text{link}(\text{root}) = \text{root}$ 。

引理 4: 所有后缀链接构成一棵根节点为初始状态的树。

证明: 考虑任意不是初始状态的状态 p , 后缀链接 $\text{link}(p)$ 连接到的状态对应于严格更短的字符串 (后缀链接的定义、引理 3)。因此, 沿后缀链接移动, 我们总是能到达对应空串的状态。

引理 5: 通过集合构造的树 (每个子节点的 LCP 都包含在父节点的 LCP 中) 与通过后缀链接构造的树相同。

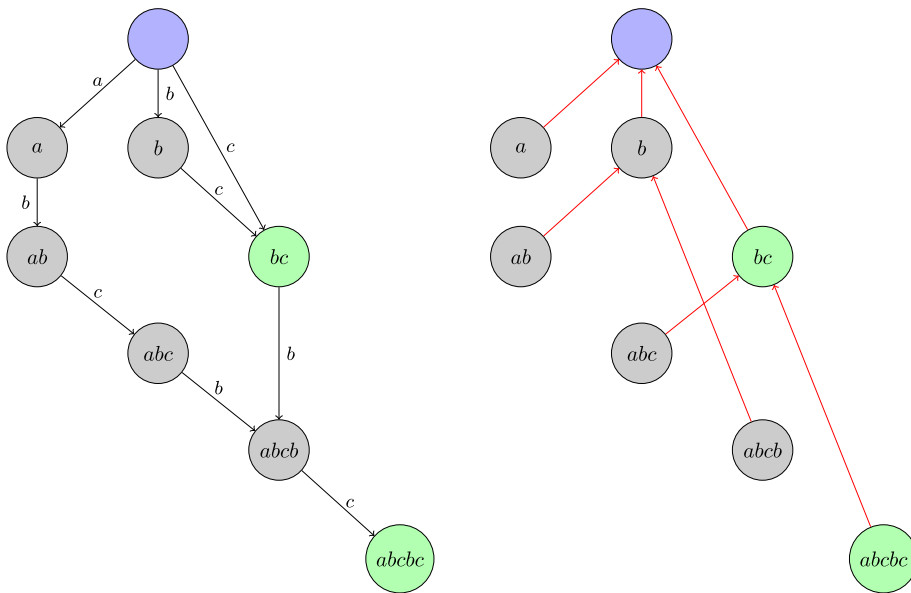
证明: 由引理 2, 任意一个 SAM 的集合形成了一棵树 (因为两个集合要么完全没有交集要么其中一个是另一个的子集)。

我们现在考虑任意不是初始状态的状态 p 及后缀链接 $\text{link}(p)$, 由后缀链接和引理 2, 我们可以得到

注意这里应该是 $\text{link}(p)$ 而不是 p , 因为若 p 是初始状态, 那么 p 和 $\text{link}(p)$ 应该被合并为一个节点。

结合前面的引理有: 后缀链接构成的树本质上是集合构成的一棵树。

以下是对字符串 u 构造 SAM 时产生的后缀链接树的一个例子, 节点被标记为对应等价类中最长的子串。



小结

在学习算法本身前，我们总结一下之前学过的知识，并引入一些辅助记号。

- 的子串可以根据它们结束的位置 被划分为多个等价类；
- SAM 由初始状态 和与每一个 等价类对应的每个状态组成；
- 对于每一个状态，一个或多个子串与之匹配。我们记 为其中最长的一个字符串，记 为它的长度。类似地，记 为最短的子串，它的长度为 。那么对应这个状态的所有字符串都是字符串 的不同的后缀，且所有字符串的长度恰好覆盖区间 中的每一个整数。
- 对于任意不是 的状态，定义后缀链接为连接到对应字符串 的长度为 的后缀的一条边。从根节点 出发的后缀链接可以形成一棵树。这棵树也表示 集合间的包含关系。
- 对于 以外的状态，可用后缀链接 表达：
- 如果从任意状态 开始顺着后缀链接遍历，总会到达初始状态 。这种情况下我们可以得到一个互不相交的区间 的序列，且它们的并集形成了连续的区间 。

算法

现在我们可以学习构造 SAM 的算法了。这个算法是 在线 算法，我们可以逐个加入字符串中的每个字符，并且在每一步中对应地维护 SAM。

为了保证线性的空间复杂度，我们将只保存 和 的值和每个状态的转移列表，我们不会标记终止状态（但是我们稍后会展示在构造 SAM 后如何分配这些标记）。

一开始 SAM 只包含一个状态，编号为 （其它状态的编号为 ）。为了方便，对于状态 我们指定 、（表示虚拟状态）。

过程

现在，任务转化为实现给当前字符串添加一个字符 的过程。算法流程如下：

- 令 为添加字符 之前，整个字符串对应的状态（一开始我们设 ，算法的最后一步更新）。
- 创建一个新的状态，并将 赋值为 ，在这时 的值还未知。
- 现在我们按以下流程进行（从状态 开始）。如果还没有到字符 的转移，我们就添加一个到状态 的转移，遍历后缀链接。如果在某个点已经存在到字符 的转移，我们就停下来，并将这个状态标记为 。
- 如果没有找到这样的状态，我们就到达了虚拟状态，我们将 赋值为 并退出。

- 假设现在我们找到了一个状态，其可以通过字符 a 转移。我们将转移到的状态标记为 q 。
- 现在我们分类讨论两种状态，要么 q ，要么不是。
- 如果 q ，我们只要将 q 赋值为 q 并退出。
- 否则就会有些复杂。需要 **复制** 状态：我们创建一个新的状态 q' ，复制 q 的除了 a 的值以外的所有信息（后缀链接和转移）。我们将 q' 赋值为 q 。
- 复制之后，我们将后缀链接从 q 指向 q ，也从 q 指向 q' 。
- 最终我们需要使用后缀链接从状态 q 往回走，只要存在一条通过 a 到状态 q 的转移，就将该转移重定向到状态 q' 。
- 以上三种情况，在完成这个过程之后，我们将 q 的值更新为状态 q' 。

如果我们还想知道哪些状态是 **终止状态** 而哪些不是，我们可以在为字符串 w 构造完完整的 SAM 后找到所有的终止状态。为此，我们从对应整个字符串的状态（存储在变量 p 中），遍历它的后缀链接，直到到达初始状态。我们将所有遍历到的节点都标记为终止节点。容易理解这样做我们会准确地标记字符串 w 的所有后缀，这些状态都是终止状态。

在下一部分，我们将详细叙述算法每一步的细节，并证明它的 **正确性**。因为我们只为 w 的每个字符创建一个或两个新状态，所以 SAM 只包含 **线性个** 状态。

而线性规模的转移个数，以及算法总体的线性运行时间还不那么清楚。

正确性证明

- 若一个转移 a 满足，则我们称这个转移是 **连续的**。否则，即当 a 时，这个转移被称为 **不连续的**。从算法描述中可以看出，连续的、不连续的转移是算法的不同情况。连续的转移是固定的，我们不会再改变了。与此相反，当向字符串中插入一个新的字符时，不连续的转移可能会改变（转移边的端点可能会改变）。
 - 为了避免引起歧义，我们记向 SAM 中插入当前字符 a 之前的字符串为 w 。
 - 算法从创建一个新状态 q 开始，对应于整个字符串 w 。我们创建一个新的节点的原因很清楚。与此同时我们也创建了一个新的字符和一个新的等价类。
 - 在创建一个新的状态之后，我们会从对应整个字符串 w 的状态通过后缀链接进行遍历。对于每一个状态，我们尝试添加一个通过字符 a 到新状态 q 的转移。然而我们只能添加与原有转移不冲突的转移。因此我们只要找到已存在的 a 的转移，我们就必须停止。
 - 最简单的情况是我们到达了虚拟状态 q_{null} ，这意味着我们为所有 a 的后缀添加了 a 的转移。这也意味着，字符 a 从未在字符串 w 中出现过。因此 q_{null} 的后缀链接为状态 q 。
 - 第二种情况下，我们找到了现有的转移 a 。这意味着我们尝试向自动机内添加一个 **已经存在的** 字符串（其中 a 为的一个后缀，且字符串 w 已经作为 a 的一个子串出现过了）。因为我们假设字符串 w 的自动机的构造是正确的，我们不应该在这里添加一个新的转移。然而，难点在于，从状态 q 出发的后缀链接应该连接到哪个状态呢？我们要把后缀链接连到一个状态上，且其中最长的一个字符串恰好是 w ，即这个状态的 a 应该是 w 。然而还不存在这样的状态，即 w 。这种情况下，我们必须通过拆开状态 q 来创建一个这样的状态。
 - 如果转移 a 是连续的，那么 w 。在这种情况下一切都很简单。我们只需要将 q 的后缀链接指向状态 q 。
 - 否则转移 a 是不连续的，即 w ，这意味着状态 q 不只对应于长度为 w 的后缀，还对应于 w 的更长的子串。除了将状态拆成两个子状态以外我们别无他法，所以第一个子状态的长度就是 w 了。
- 我们如何拆开一个状态呢？我们 **复制** 状态 q ，产生一个状态 q' ，我们将 q' 赋值为 q 。由于我们不想改变遍历到 q 的路径，我们将 q 的所有转移复制到 q' 。我们也将从 q 出发的后缀链接设置为 q' 的后缀链接的目标，并设置 q 的后缀链接为 q' 。
- 在拆开状态后，我们将从 q 出发的后缀链接设置为 q' 。
- 最后一步我们将一些到 q 转移重定向到 q' 。我们需要修改哪些转移呢？只重定向相当于所有字符串 w （其中 w 是 w 的最长字符串）的后缀就够了。即，我们需要继续沿着后缀链接遍历，从结点 q 直到虚拟状态 q_{null} 或者是转移到不是状态 q 的一个转移。

对操作次数为线性的证明

首先我们假设字符集大小为 **常数**。如果字符集大小不是常数，SAM 的时间复杂度就不是线性的。从一个结点出发的转移存储在支持快速查询和插入的平衡树中。因此如果我们记 n 为字符集， m 为字符串大小，则算法的渐进时间复杂度为 $O(nm)$ ，空间复杂度为 $O(nm)$ 。然而如果字符集足够小，可以不写平衡树，以空间换时间将每个结点的转移存储为长度为 n 的数组（用于快速查询）和链表（用于快速遍历所有可用关键字）。这样算法的时间复杂度为 $O(nm)$ ，空间复杂度为 $O(nm)$ 。

所以我们将认为字符集的大小为常数，即每次对一个字符搜索转移、添加转移、查找下一个转移。这些操作的时间复杂度都为。

如果我们考虑算法的各个部分，算法中有三处时间复杂度不明显是线性的：

- 第一处是遍历所有状态 的后缀链接，添加字符 的转移。
- 第二处是当状态 被复制到一个新的状态 时复制转移的过程。
- 第三处是修改指向 的转移，将它们重定向到 的过程。

我们使用 SAM 的大小（状态数和转移数）为 **线性的** 的事实（对状态数是线性的的证明就是算法本身，对转移数为线性的的证明将在稍后实现算法后给出）。

因此上述 **第一处和第二处** 的总复杂度显然为线性的，因为单次操作均摊只为自动机添加了一个新转移。

还需为 **第三处** 估计总复杂度，我们将最初指向 的转移重定向到 。我们记，这是一个字符串 的后缀，每次迭代长度都递减——因为字符串 的位置每次迭代都单调上升。这种情况下，如果在循环的第一次迭代之前，相对应的字符串在距离 的深度为 的位置上（深度记为后缀链接的数量），那么在最后一次迭代后，字符串 将会成为路径上第二个从出发的后缀链接（它将会成为新的 的值）。

因此，循环中的每次迭代都会使作为当前字符串的后缀的字符串 的位置单调递增。因此这个循环最多不会执行超过次迭代，这正是我们需要证明的。

实现

首先，我们实现一种存储一个转移的全部信息的数据结构。如果需要的话，你可以在这里加入一个终止标记，也可以是一些其它信息。我们将用一个 `map` 存储转移的列表，允许我们在总计 的空间复杂度和 的时间复杂度内处理整个字符串。（注：在字符集大小为较小的常数，比如 26 时，将 `next` 定义为 `int[26]` 更方便）

```
1 struct state {
2     int len, link;
3     std::map<char, int> next;
4 };
```

SAM 本身将会存储在一个 `state` 结构体数组中。我们记录当前自动机的大小 `sz` 和变量 `last`，当前整个字符串对应的状态。

```
1 constexpr int MAXLEN = 100000;
2 state st[MAXLEN * 2];
3 int sz, last;
```

我们定义一个函数来初始化 SAM（创建一个只有初始状态的 SAM）。

```
1 void sam_init() {
2     st[0].len = 0;
3     st[0].link = -1;
4     sz++;
5     last = 0;
6 }
```

最终我们给出主函数的实现：给当前行末增加一个字符，对应地在之前的基础上建造自动机。

▼ 实现

```

1 void sam_extend(char c) {
2     int cur = sz++;
3     st[cur].len = st[last].len + 1;
4     int p = last;
5     while (p != -1 && !st[p].next.count(c)) {
6         st[p].next[c] = cur;
7         p = st[p].link;
8     }
9     if (p == -1) {
10        st[cur].link = 0;
11    } else {
12        int q = st[p].next[c];
13        if (st[p].len + 1 == st[q].len) {
14            st[cur].link = q;
15        } else {
16            int clone = sz++;
17            st[clone].len = st[p].len + 1;
18            st[clone].next = st[q].next;
19            st[clone].link = st[q].link;
20            while (p != -1 && st[p].next[c] == q) {
21                st[p].next[c] = clone;
22                p = st[p].link;
23            }
24            st[q].link = st[cur].link = clone;
25        }
26    }
27    last = cur;
28 }

```

正如之前提到的一样，如果你用内存换时间（空间复杂度为 $O(n \cdot |\Sigma|)$ ，其中 n 为字符串大小），你可以在 $O(n)$ 的时间内构造字符集大小任意的 SAM。但是这样你需要为每一个状态储存一个大小为 $|\Sigma|$ 的数组（用于快速跳转到转移的字符），和另外一个所有转移的链表（用于快速在转移中迭代）。

更多性质

状态数

对于一个长度为 n 的字符串，它的 SAM 中的状态数 **不会超过** $2n$ （假设 $|\Sigma| \geq 2$ ）。

算法本身即可证明该结论。一开始，自动机含有一个状态，第一次和第二次迭代中只会创建一个节点，剩余的 $n-2$ 步中每步会创建至多 2 个状态。

然而我们也能在 **不借助这个算法** 的情况下 **证明** 这个估计值。我们回忆一下状态数等于不同的子串个数。这些子串形成了一棵树（祖先节点的集合包含了它所有孩子节点的集合）。考虑将这棵树稍微变形一下：只要它有一个只有一个孩子的内部结点（这意味着该子节点的集合至少遗漏了它的父集合中的一个位置），我们创建一个含有这个遗漏位置的集合。最后我们可以获得一棵每一个内部结点的度数大于 1 的树，且叶子节点的个数不超过 n 。因此这样的树里有不超过 $2n$ 个节点。

字符串 s 的状态数达到了该上界：从第三次迭代后的每次迭代，算法都会拆开一个状态，最终产生恰好 $2n$ 个状态。

转移数

对于一个长度为 n 的字符串，它的 SAM 中的转移数 **不会超过** $2n$ （假设 $|\Sigma| \geq 2$ ）。

证明如下：

我们首先估计连续的转移的数量。考虑自动机中从状态 0 开始的所有最长路径的生成树。生成树只包含连续的边，因此数量少于状态数，即边数不会超过 $2n$ 。

现在我们来估计不连续的转移的数量。令当前不连续转移为 l ，其字符为 c 。我们取它的对应字符串 l ，其中字符串 l 对应于初始状态到 l 的最长路径， l 对应于从 l 到任意终止状态的最长路径。一方面，每个不完整的字符串所对应的形如 l 的字符串是不同的（因为字符串 l 和 l' 仅由完整的转移组成）。另一方面，由终止状态的定义，每个形如 l 的字符串都是整个字符串 l 的后缀。因为 l 只有 l 个非空后缀，且形如 l 的字符串都不包含 l （因为整个字符串只包含完整的转移），所以非完整的转移的总数不会超过 l 。

将以上两个估计值相加，我们可以得到上界 l 。然而，最大的状态数只能在类似于 l 的情况中产生，而此时转移数量显然少于 l 。

因此我们可以获得更为紧确的 SAM 的转移数的上界： l 。字符串 l 就达到了这个上界。

额外信息

观察 [实现](#) 中的结构体的每个变量。实际上，尽管 SAM 本身由 `next` 组成，但 SAM 构造算法中作为辅助变量的 `link` 和 `len` 在应用中常常比 `next` 重要，甚至可以抛开 `next` 单独使用。

设字符串的长度为 l ，考虑 `extend` 操作中 `cur` 变量的值，这个节点对应的状态是执行 `extend` 操作时的当前字符串，即字符串的一个前缀，每个前缀有一个终点。这样得到的 l 个节点，对应了 l 个不同的 **终点**。设第 i 个节点为 i ，对应的是 i ，终点是 i 。姑且把这些节点称之为「终点节点」。

考虑给 SAM 赋予树形结构，树的根为 0，且其余节点 i 的父亲为 $l[i].link$ 。则这棵树与原 SAM 的关系是：

- 每个节点的终点集合等于其 **子树** 内所有终点节点对应的终点的集合。

在此基础上可以给每个节点赋予一个最长字符串，是其终点集合中 **任意** 一个终点开始 **往前** 取 `len` 个字符得到的字符串。每个这样的字符串都一样，且 `len` 恰好是满足这个条件的最大值。

这些字符串满足的性质是：

- 如果节点 A 是 B 的祖先，则节点 A 对应的字符串是节点 B 对应的字符串的 **后缀**。

这条性质把字符串所有前缀组成了一棵树，且有许多符合直觉的树的性质。例如， l 和 l' 的最长公共后缀对应的字符串就是 l 和 l' 对应的 LCA 的字符串。实际上，这棵树与将字符串 l 翻转后得到字符串的压缩后缀树结构相同。

每个状态 i 对应的子串数量是 $l[i].len - l[i].link.len + 1$ （节点 0 例外）。注意到 $l[i].len$ 对应的字符串是 $l[i].link$ 对应的字符串的一个后缀，这些子串就是 $l[i].link$ 对应字符串的所有后缀，去掉被父亲「抢掉」的那部分，即 $l[i].link$ 对应字符串的所有后缀。

应用

下面我们来看一些可以用 SAM 解决的问题。简单起见，假设字符集的大小 Σ 为常数。这允许我们认为增加一个字符和遍历的复杂度为常数。

检查字符串是否出现

给一个文本串 T 和多个模式串 P ，我们要检查字符串 P 是否作为 T 的一个子串出现。

我们在 $O(|T|)$ 的时间内对文本串 T 构造后缀自动机。为了检查模式串 P 是否在 T 中出现，我们沿转移（边）从 1 开始根据 P 的字符进行转移。如果在某个点无法转移下去，则模式串 P 不是 T 的一个子串。如果我们能够这样处理完整个字符串 P ，那么模式串在 T 中出现过。

对于每个字符串 P ，算法的时间复杂度为 $O(|P|)$ 。此外，这个算法还找到了模式串 P 在文本串中出现的最大前缀长度。

不同子串个数

给一个字符串 S ，计算不同子串的个数。

对字符串 S 构造后缀自动机。

每个 的子串都相当于自动机中的一些路径。因此不同子串的个数等于自动机中以 为起点的不同路径的条数。

考虑到 SAM 为有向无环图，不同路径的条数可以通过动态规划计算。即令 为从状态 开始的路径数量（包括长度为零的路径），则我们有如下递推方程：

即， 可以表示为所有 的转移的末端的和。

所以不同子串的个数为 （因为要去掉空子串）。

总时间复杂度为： 。

另一种方法是利用上述后缀自动机的树形结构。每个节点对应的子串数量是 ，对自动机所有节点求和即可。

例题：[【模板】后缀自动机](#)，[SDOI2016 生成魔咒](#)

所有不同子串的总长度

给定一个字符串 ，计算所有不同子串的总长度。

本题做法与上一题类似，只是现在我们需要考虑分两部分进行动态规划：不同子串的数量 和它们的总长度 。

我们已经在上一题中介绍了如何计算 。 的值可以通过以下递推式计算：

我们取每个邻接结点 的答案，并加上 （因为从状态 出发的子串都增加了一个字符）。

算法的时间复杂度仍然是 。

同样可以利用上述后缀自动机的树形结构。每个节点对应的所有后缀长度是 ，减去其 节点的对应值就是该节点的净贡献，对自动机所有节点求和即可。

字典序第 k 大子串

给定一个字符串 。多组询问，每组询问给定一个数 ，查询 的所有子串中字典序第 大的子串。

这个问题的思路可以从解决前两个问题的思路发展而来。字典序第 大的子串对应于 SAM 中字典序第 大的路径，因此在计算每个状态的路径数后，我们可以很容易地从 SAM 的根开始找到第 大的路径。

预处理的时间复杂度为 ，单次查询的复杂度为 （其中 是查询的答案， 为字符集的大小）。

虽然该题是后缀自动机的经典题，但实际上这题由于涉及字典序，用后缀数组做最方便。

例题：[SPOJ - SUBLEX](#)，[TJOI2015 弦论](#)

最小循环移位

给定一个字符串 。找出字典序最小的循环移位。

容易发现字符串 包含字符串 的所有循环移位作为子串。

所以问题简化为在 对应的后缀自动机上寻找最小的长度为 的路径，这可以通过平凡的方法做到：我们从初始状态开始，贪心地访问最小的字符即可。

总的时间复杂度为 。

出现次数

对于一个给定的文本串，有多组询问，每组询问给一个模式串，回答模式串在字符串中作为子串出现了多少次。

利用后缀自动机的树形结构，进行 dfs 即可预处理每个节点的终点集合大小。在自动机上查找模式串对应的节点，如果存在，则答案就是该节点的终点集合大小；如果不存在，则答案为 0。

以下为原方法：

对文本串构造后缀自动机。

接下来做预处理：对于自动机中的每个状态，预处理，使之等于集合的大小。事实上，对应同一状态的所有子串在文本串中的出现次数相同，这相当于集合中的位置数。

然而我们不能明确的构造集合，因此我们只考虑它们的大小。

为了计算这些值，我们进行以下操作。对于每个状态，如果它不是通过复制创建的（且它不是初始状态），我们将它的初始化为 1。然后我们按它们的长度降序遍历所有状态，并将当前的值加到后缀链接指向的状态上，即：

这样做每个状态的答案都是正确的。

为什么这是正确的？不是通过复制获得的状态，恰好有 n 个，并且它们中的前 n 个在我们插入前 n 个字符时产生。因此对于每个这样的状态，我们在它被处理时计算它们所对应的位置的数量。因此我们初始将这些状态的 val 赋为 1，其它状态的 val 赋为 0。

接下来我们对每一个执行以下操作：。其背后的含义是，如果有一个字符串出现了 cnt 次，那么它的所有后缀也在完全相同的地方结束，即也出现了 cnt 次。

为什么我们在这个过程中不会重复计数（即把某些位置数了两次）呢？因为我们只将一个状态的位置添加到一个其它的状态上，所以一个状态不可能以两种不同的方式将其位置重复地指向另一个状态。

因此，我们可以在 $O(n)$ 的时间内计算出所有状态的 val 值。

最后回答询问只需要查找值，其中 st 为模式串对应的状态，如果该模式串不存在答案就为 0。单次查询的时间复杂度为 $O(|s|)$ 。

第一次出现的位置

给定一个文本串，多组查询。每次查询字符串在字符串中第一次出现的位置（的开头位置）。

我们构造一个后缀自动机。我们对 SAM 中的所有状态预处理位置。即，对每个状态 我们想要找到第一次出现这个状态的末端的位置。换句话说，我们希望先找到每个集合中的最小的元素（显然我们不能显式地维护所有集合）。

为了维护这些位置，我们将原函数扩展为 `sam_extend()`。当我们创建新状态时，我们令：

；当我们把结点复制到 st 时，我们令：

（因为值的唯一的其它选项显然太大了）。

那么查询的答案就是 $pos[st]$ ，其中 st 为对应字符串的状态。单次查询只需要 $O(1)$ 的时间。

所有出现的位置

问题同上，这一次需要查询文本串 中模式串出现的所有位置。

利用后缀自动机的树形结构，遍历子树，一旦发现终点节点就输出。

以下为原解法：

我们还是对文本串 构造后缀自动机。与上一个问题相似，我们为所有状态计算位置 。

如果 为对应于模式串 的状态，显然 为答案的一部分。需要查找的其它位置怎么办？我们使用了含有字符串 的自动机，我们还需要将哪些状态纳入自动机呢？所有对应于以 为后缀的字符串的状态。换句话说我们要找到所有可以通过后缀链接到达状态 的状态。

因此为了解决这个问题，我们需要为每一个状态保存一个指向它的后缀引用列表。查询的答案就包含了对于每个我们能从状态 只使用后缀引用进行 DFS 或 BFS 的所有状态的 值。

这种变通方案的时间复杂度为 ，因为我们不会重复访问一个状态（因为对于仅有一个后缀链接指向一个状态，所以不存在两条不同的路径指向同一状态）。

我们只需要考虑两个可能有相同 值的不同状态。如果一个状态是由另一个复制而来的，则这种情况会发生。然而，这并不会对复杂度分析造成影响，因为每个状态至多被复制一次。

此外，如果我们不从被复制的节点输出位置，我们也可以去除重复的位置。事实上对于一个状态，如果经过被复制状态可以到达，则经过原状态也可以到达。因此，如果我们给每个状态记录标记 `is_clone` 来代表这个状态是不是被复制出来的，我们就可以简单地忽略掉被复制的状态，只输出其它所有状态的 的值。

以下是大致的实现：

```
1 struct state {
2     bool is_clone;
3     int first_pos;
4     std::vector<int> inv_link;
5     // some other variables
6 };
7
8 // 在构造 SAM 后
9 for (int v = 1; v < sz; v++) st[st[v].link].inv_link.push_back(v);
10
11 // 输出所有出现位置
12 void output_all_occurrences(int v, int P_length) {
13     if (!st[v].is_clone) cout << st[v].first_pos - P_length + 1 << endl;
14     for (int u : st[v].inv_link) output_all_occurrences(u, P_length);
15 }
```

最短的没有出现的字符串

给定一个字符串 和一个特定的字符集，我们要找一个长度最短的没有在 中出现过的字符串。

我们在字符串 的后缀自动机上做动态规划。

令 为节点 的答案，即，我们已经处理完了子串的一部分，当前在状态 ，想找到不连续的转移需要添加的最小字符数量。计算 非常简单。如果不存在使用字符集中至少一个字符的转移，则 。否则添加一个字符是不够的，我们需要求出所有转移中的最小值：

问题的答案就是 ，字符串可以通过计算过的数组 逆推回去。

两个字符串的最长公共子串

给定两个字符串 S 和 T ，求出最长公共子串，公共子串定义为在 S 和 T 中都作为子串出现过的字符串。

我们对字符串 S 构造后缀自动机。

我们现在处理字符串 T ，对于每一个前缀，都在 S 中寻找这个前缀的最长后缀。换句话说，对于每个字符串 T 中的位置，我们想要找到这个位置结束的 S 和 T 的最长公共子串的长度。显然问题的答案就是所有 T 的最大值。

为了达到这一目的，我们使用两个变量，**当前状态** 和 **当前长度**。这两个变量描述当前匹配的部分：它的长度和它们对应的状态。

一开始 $v = 0$ 且 $l = 0$ ，即，匹配为空串。

现在我们来描述如何添加一个字符 c 并为其重新计算答案：

- 如果存在一个从 v 到字符 c 的转移，我们只需要转移并让 l 自增一。
- 如果不存在这样的转移，我们需要缩短当前匹配的部分，这意味着我们需要按照后缀链接进行转移：

与此同时，需要缩短当前长度。显然我们需要将 l 赋值为 $l - 1$ ，因为经过这个后缀链接后我们到达的状态所对应的最长字符串是一个子串。

- 如果仍然没有使用这一字符的转移，我们继续重复经过后缀链接并减小 l ，直到我们找到一个转移或到达虚拟状态（这意味着字符 c 根本没有在 S 中出现过，所以我们设置 $l = 0$ ）。

这一部分的时间复杂度为 $O(|T|)$ ，因为每次移动我们要么可以使 l 增加一，要么可以在后缀链接间移动几次，每次都减小 l 的值。

代码实现：

```
1 string lcs(const string &S, const string &T) {
2     sam_init();
3     for (int i = 0; i < S.size(); i++) sam_extend(S[i]);
4
5     int v = 0, l = 0, best = 0, bestpos = 0;
6     for (int i = 0; i < T.size(); i++) {
7         while (v && !st[v].next.count(T[i])) {
8             v = st[v].link;
9             l = st[v].length;
10        }
11        if (st[v].next.count(T[i])) {
12            v = st[v].next[T[i]];
13            l++;
14        }
15        if (l > best) {
16            best = l;
17            bestpos = i;
18        }
19    }
20    return T.substr(bestpos - best + 1, best);
21 }
```

例题：[SPOJ Longest Common Substring](#)

多个字符串间的最长公共子串

给定 n 个字符串 $S_1, S_2, \dots, S_n$ 。我们需要找到它们的最长公共子串，即作为子串出现在每个字符串中的字符串。

我们将所有的子串连接成一个较长的字符串 S ，以特殊字符 $\$$ 分开每个字符串（一个字符对应一个字符串）：

然后对字符串 构造后缀自动机。

现在我们需要在自动机中找到存在于所有字符串 中的一个字符串，这可以通过使用添加的特殊字符完成。注意如果包含了一个子串，则 SAM 中存在一条从包含字符 的子串而不包含以其它字符 开始的路径。

因此我们需要计算可达性，即对于自动机中的每个状态和每个字符，是否存在这样的一条路径。这可以容易地通过 DFS 或 BFS 及动态规划计算。之后，问题的答案就是状态 的字符串 中存在所有特殊字符的路径。

例题： [SPOJ Longest Common Substring II](#)

例题

- [HihoCoder #1441：后缀自动机一·基本概念](#)
- [【模板】后缀自动机](#)
- [SDOI2016 生成魔咒](#)
- [SPOJ - SUBLEX](#)
- [TJOI2015 弦论](#)
- [SPOJ Longest Common Substring](#)
- [SPOJ Longest Common Substring II](#)
- [Codeforces 1037H Security](#)
- [Codeforces 666E Forensic Examination](#)
- [HDU4416 Good Article Good sentence](#)
- [HDU4436 str2int](#)
- [HDU6583 Typewriter](#)
- [Codeforces 235C Cyclical Quest](#)
- [CTSC2012 熟悉的文章](#)
- [NOI2018 你的名字](#)

相关资料

我们先给出与 SAM 有关的最初的一些文献：

- A. Blumer, J. Blumer, A. Ehrenfeucht, D. Haussler, R. McConnell. **Linear Size Finite Automata for the Set of All Subwords of a Word. An Outline of Results**[1983]
- A. Blumer, J. Blumer, A. Ehrenfeucht, D. Haussler. **The Smallest Automaton Recognizing the Subwords of a Text**[1984]
- Maxime Crochemore. **Optimal Factor Transducers**[1985]
- Maxime Crochemore. **Transducers and Repetitions**[1986]
- A. Nerode. **Linear automaton transformations**[1958]

另外，在更新的一些资源以及很多关于字符串算法的书中，都能找到这个主题：

- Maxime Crochemore, Rytter Wowjcieh. **Jewels of Stringology**[2002]
- Bill Smyth. **Computing Patterns in Strings**[2003]
- Bill Smith. **Methods and algorithms of calculations on lines**[2006]

另外，还有一些资料：

- 《后缀自动机》，陈立杰。
- 《后缀自动机在字典树上的拓展》，刘研绎。
- 《后缀自动机及其应用》，张天扬。
- <https://www.cnblogs.com/zinthos/p/3899679.html>
- <https://codeforces.com/blog/entry/20861>

- <https://zhuanlan.zhihu.com/p/25948077>

本页面主要译自博文 [Суффиксный автомат](#) 与其英文翻译版 [Suffix Automaton](#)。其中俄文版版权协议为 **Public Domain + Leave a Link**；英文版版权协议为 **CC-BY-SA 4.0**。

本页面最近更新：2024/10/9 22:38:42, [更新历史](#)

发现错误？想一起完善？ [在 GitHub 上编辑此页！](#)

本页面贡献者：[Chrogeek](#), [GoodCoder666](#), [ksyx](#), [abc1763613206](#), [Backlight](#), [CCXXI](#), [ChungZH](#), [countercurrent-time](#), [Dev-XYs](#), [Enter-tainer](#), [FFjet](#), [frank-xjh](#), [fstqwq](#), [GeZiyue](#), [Ghastlcon](#), [H-J-Granger](#), [Heartfirey](#), [Henry-ZHR](#), [iamtwz](#), [interestingLSY](#), [Ir1d](#), [Kinandra](#), [LeoJacob](#), [longlongzhu123](#), [Luckyblock233](#), [mao1t](#), [Marcythm](#), [mgt](#), [MingqiHuang](#), [opsiff](#), [ouuan](#), [ranwen](#), [shuzhouliu](#), [sshwy](#), [SukkaW](#), [Suyun514](#), [Tiphereth-A](#), [vincent-163](#), [WhenMelancholy](#), [Xeonacid](#), [yjl9903](#), [ZXyaang](#)

本页面的全部内容 [在 CC BY-SA 4.0 和 SATA 协议之条款下](#) 提供，附加条款亦可能应用